

Detecting and adapting to crisis pattern with context based Deep Reinforcement Learning

Eric Benhamou¹, David Saltiel^{2,3}, Jean-Jacques Ohana⁴, and Jamal Atif¹

¹MILES, Machine Learning Group, LAMSADE, Dauphine, France ²Ai Square Connect, Research group, France

³LISIC, ULCO, France ⁴Multi Assets Solutions, Homa Capital, France

¹Email: {eric.benhamou, jamal.atif}@lamsade.dauphine.fr, ²Email: david.saltiel@aisquareconnect.com

⁴Email: jjohana@homacapital.fr

Abstract—Deep reinforcement learning (DRL) has reached super human levels in complex tasks like game solving (Go [1], StarCraft II [2]), and autonomous driving [3]. However, it remains an open question whether DRL can reach human level in applications to financial problems and in particular in detecting pattern crisis and consequently dis-investing. In this paper, we present an innovative DRL framework consisting in two sub-networks fed respectively with portfolio strategies past performances and standard deviations as well as additional contextual features. The second sub network plays an important role as it captures dependencies with common financial indicators features like risk aversion, economic surprise index and correlations between assets that allows taking into account context based information. We compare different network architectures either using layers of convolutions to reduce network's complexity or LSTM block to capture time dependency and whether previous allocations is important in the modeling. We also use adversarial training to make the final model more robust. Results on test set show this approach substantially over-performs traditional portfolio optimization methods like Markovitz and is able to detect and anticipate crisis like the current Covid one.

I. INTRODUCTION

Being able to adapt portfolio allocation to crisis environment like the current Covid crisis is a major concern for the financial industry. Indeed, the current Covid crisis took the industry by surprise twice. First, when stock markets plunged at an unprecedented speed in March 2020 with the SP 500 falling by 13 %, asset managers were slow to react and to cut risk exposure. And secondly, when stock markets bounced back up at an equally rapid pace, with a rise of 13 % for the SP 500 in May 2020, asset managers were again overhauled. In contrast, the previous 2008 crisis was very slow both in terms of its falls and recovery. Hence, adapting portfolio allocation to crisis environment is a very important matter and has attracted growing attention from the financial scientific community.

The standard approach for portfolio allocation, that serves as a base line for our research, relies on determining portfolio weights according to a risk return criterion. The so called Markovitz portfolio [4] finds the optimal allocation by determining the portfolio with minimum variance given a target return or equivalently the portfolio with maximum return given a targeted level of variance (the dual optimization). However, this approach suffers from a major flaw because of unreliable risk estimations of the individual portfolio strategy excess returns and covariances. This leads not only to unstable al-

locations, but also to slow reactions to changing environments [5]. If we want to find a more dynamic allocation method, deep reinforcement learning is an appealing method. It reformulates the portfolio optimization problem as a continuous control program with delayed rewards. Rules are simple. Each trading day, the dynamic virtual asset manager agent has the right to modify the portfolio allocation. When it modifies the portfolio weights, it incurred transaction costs. The agent can only allocate between 0 and 100 % for all the portfolio assets. It can not short any asset, hence weights are always positive and never above 100 %. It can not neither borrow to fund leverage positions, hence the sum of all allocations is strictly equal to 100 %. To make decisions, the dynamic agent has access not only to past performances but also some financial contextual information that helps it making an informed decision. The agent receives in terms of feedback a financial reward that orientates its decisions. Compared to traditional financial methods, this approach has the major advantage to adapt to changing market conditions and to be somehow more model free than traditional financial methods as we connect portfolio allocations directly to financial data and not to specific risk factors that may factor in some cognitive bias. This stream of research is also highly motivated by the recent major progress of deep reinforcement learning methods that have reached super human levels in complex tasks like game solving (historically Atari games [6], Go [1], StarCraft II [2]), and autonomous driving [3]. Nonetheless, it still remains an open question whether DRL can reach human level in applications to financial problems and in particular in detecting pattern crisis and consequently dis-investing.

A. Related Work

Initially, many of the machine-learning and in particular deep network applications to financial markets tried to predict price movements or trends [7], [8], [9]. The logic was to take historical prices of assets as inputs and use deep neural networks to predict asset prices for the next period. Armed with the forecast, a trading agent can act and decide the best allocation. The problem to solve is a standard supervised learning task and more precisely a regression problem. It is straightforward to implement. Yet, the efficiency of the method relies heavily on the accuracy of the prediction, which makes the method quite fragile and questionable as future market

prices are well known to be difficult to predict. Furthermore, this approach tends to reduce substantially portfolio diversification and can not cope easily with transactions costs. In contrast, DRL can easily tackle these issues as it does not aim at predicting prices but rather at finding the optimal action or for our matter the optimal allocation.

The idea of applying DRL to portfolio allocation has recently taking off in the machine learning community with some recent works on crypto currencies [10], [11], [12], [13] and [14]. Compared to traditional approaches on financial time series, that aim at taking decision based on forecasting estimates, [11] and [12] showed that deep reinforcement learning with Convolutional Neural Network (CNN) architecture tends to perform better for crypto currencies and Chinese stock markets than deep learning architecture that relies on time series forecast like LSTM. However, when there is a very rapid crisis, like what happened during the Covid crisis, using just past performances may lead the DRL agent to react too slowly. To make an analogy, it is as if the agent was self-driving on the highway and very brutally, an obstacle arises. Using past performances only is like looking in the mirrors behind to infer what will happen next. Adding a context is like lifting up our eyes and looking further forward. Context based reinforcement learning has recently emerged as strong tool to increase reinforcement learning dynamic agent performance [15], [16]. More specifically, context based reinforcement learning (RL) with high capacity function approximators, such as deep neural networks (DNNs), has in the last two years attracted growing attention and been the subject of many publications in notorious machine learning conferences as it solved efficiently a variety of sequential decision-making problems, including board games (e.g., Go and Chess ([17]), video games (e.g., Atari games ([18]), and complex robotic control tasks ([19], [20], [21]). Theoretically, it has also been advocated that the usage of a context enables achieving superior data-efficiency to model-free RL methods in general [22], [23].

So in this work, we extend previous works of DRL by precisely using a context based approach. This is done by integrating common financial states in our deep network, having at least two sub networks and potentially three if we also incorporate in states the previous allocations. Experiments show that this approach is able to pick the best portfolio allocation out of sample using financial features used by asset managers: risk aversion index, correlation between equities and bonds, Citi economic surprise index and to accommodate for crisis by reducing risk exposure. We provide performances out of sample and test various configurations to emphasize that using CNN works much better than more predictive architecture like LSTM confirming previous works.

B. Contributions

Our contributions are twofold:

- First we explain why a context based deep reinforcement learning approach is closer to human thinking and leads to better results, with a novel deep network architecture consisting of two sub networks: one network (network

1) that takes as inputs past performances (and standard deviation) of the portfolio strategies and another one (network 2) that takes as inputs financial contextual information related to the performances of the portfolio strategies that are thought to have some predictive power regarding portfolio strategies future performances.

- Second, we summarize lots of empirical findings. Reward function is critical. Sharpe ratio reward leads to different results compared to a straight final net performance reward function. CNN performs better than LSTM and captures implicit features. Using adversarial training by adding noise to the data improves the model. Last but not least, dependency to previous allocations does not improve the model.

II. MATHEMATICAL FORMULATION

As summarized by figure 1, an asset manager robot has several strategies that it wants to allocate optimally, with a performance objective on the overall portfolio. Not only does it have access to historical daily performance (the middle rectangle in figure 1) but it can also leverage additional information (the rectangle on the left in figure 1) that provides some contextual information about market conditions. These are other price data points but also unstructured data like some macro economic data. To gauge the performance of its decision, it has an objective that can be either the net performance of the portfolio or some risk return criterion (the third rectangle of figure 1 on the right)

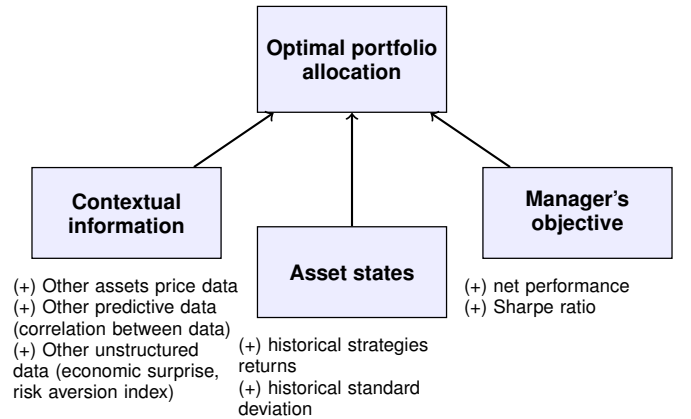


Fig. 1. Portfolio allocation problem

The question of the asset allocation can be reformulated as a standard reinforcement learning problem, thanks to Markov Decision Process (MDP). The learning agent interacts with an environment E to decide rational or optimal actions and receives in return some rewards. These rewards are not necessarily only positive and are given only at the end of the financial episode. These rewards act as a feedback for finding the best action. Using the established formalism of Markov decision process, we assume that there exists a discrete time stochastic control process represented by a 4-tuple defined by $(\mathcal{S}, \mathcal{A}, P_a, R_a)$ where $\mathcal{S}$ is the set of states, $\mathcal{A}$ the set of actions, $P_a(s, s') = \Pr(s_{t+1} = s' \mid s_t = s, a_t = a)$ the

transition probability that action a in state s at time t will lead to state s' at the next period $t + 1$ and finally, $R_a(s, a)$ the immediate reward received after state s and action a .

The requirement of a Markovian state that guarantees that there exists a solution (hence satisfying the Bellman optimality principle [24]) is a strong assumption that is hard to verify in practice. It is somehow levied in practice by stacking enough observations to enforce that the Markov property is satisfied. Hence, it is useful, following [25] or [26], to introduce the concept of observations and pile them to coin states. In this setting, the agent perceives at time t an observation o_t along with a reward r_t .

In our setting, time is divided into trading periods of equal length τ . In the rest of the paper, τ represents one trading day but the setting can be applied to shorter time periods, like 30 minutes, to deal with intraday trading decisions. At the beginning of each trading period, a trading robot decides to potentially reallocate the funds among m assets. The trading robot has access to an environment that provides at each time t , a state s_t that is composed of the past observations that are rich enough to assume Markovianity. Intuitively, it is important for the agent to observe not only the last returns but also some previous returns (like the returns over 2, 3 and 4 business days, but also a week and potentially a month) to make a decision. Mathematically, we denote by δ_1 the lag operator applied to each observation. To make this concrete the lag operator δ_1 's outputs are the last portfolio strategy returns at time t but also at time $t - 1$, $t - 2$, $t - 3$ and so on. There is here some trade-off. We obviously need enough observations to mimic a Markovian setting to ensure problem is well posed. But we also need to reduce observations to avoid facing the curse of dimensionality. We will discuss this point in our experience, but practically, we take returns at time $t - 60$ representing returns 3 months ago, $t - 20$ one month ago¹, $t - 4$, $t - 3$, $t - 2$, $t - 1$ and t , the latter four providing returns over the last trading week.

By abuse of language, we can represent the lag δ_1 operator by a vector of lagging periods $\delta_1 = [0, 1, 2, 3, 4, 20, 60]$ (as there is a one to one mapping between the operator and the lagging periods) and retrieve the corresponding returns for asset i as follows: $[r_t^i, r_{t-1}^i, r_{t-2}^i, \dots, r_{t-30}^i, r_{t-60}^i]$. Inputs that we call asset states as they directly relate to the portfolio's assets are not only past returns lagged over the δ_1 periods but also standard deviation. The intuition behind the consumption of returns standard deviation or equivalently their volatility is that volatility is a good predictor of crisis. Indeed it is a stylized fact in the financial literature that volatility is a good predictor of risk. [27] [28] and that an increase of volatility comes swiftly after a market crash [29] [30]. The period to compute the volatility is a hyper parameter, again another hyper-parameter to fine-tune and is arbitrarily taken

to 20 periods to represent a month of data. If we summarize, asset states A_t are given by two matrices $A_t = [A_t^1, A_t^2]$ with the first matrix A_t^1 (in red) being the matrix of returns:

$$A_t^1 = \begin{pmatrix} r^1(t) & \dots & r^1(t-60) \\ \dots & \dots & \dots \\ r^m(t) & \dots & r^m(t-60) \end{pmatrix}$$

while the second matrix (in blue) A_t^2 containing standard deviations:

$$A_t^2 = \begin{pmatrix} \sigma^1(t) & \dots & \sigma^1(t-60) \\ \dots & \dots & \dots \\ \sigma^m(t) & \dots & \sigma^m(t-60) \end{pmatrix}$$

The asset states are stored in a 3-D tensor as shown in figure 2. Its similarities with image where pixels are stored in 3 different matrices representing red, green and blue image enable us to use 2 dimensional convolution network for our deep network. The analogy goes even further as it is well known in image recognition that convolutional networks achieves strong performances thanks to their capacity to extract meaningful features and to have very limited parameters hence avoiding over-fitting.

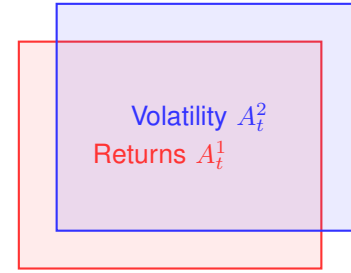


Fig. 2. 3 dimensional tensor: the asset states

To introduce conceptual based information, the asset manager robot observes also additional important features denoted by C_t that provides insights about the future evolution of the portfolio strategies. Using market knowledge from Homa capital multi assets solutions, we add 3 features (referred to as contextual features) that are correlation between equity and bonds denoted by c_t^1 , Citigroup global economic surprise index denoted by c_t^2 , and risk aversion index denoted by c_t^3 . These features are not taken at random but are well known or at least assumed to have some predictive power for our portfolio strategies as these strategies incorporates a mix of equity and bonds and are highly sensitive to economic surprise and risk aversion level. Again to ensure somehow some Markovianity and to include in the current knowledge of the virtual agent more than the last observation of these features, we introduce a second lag operator δ_2 that operates on the contextual features. To keep things simple in our experience, we take

¹as there are approximately 60 trading days in a quarter, and 20 days in a month

the same vector of lagging periods to represent this second lag operator although the method can be fine-tuned with two different lags for the asset and contextual states. In our setting, $\delta_2 = [0, 1, 2, 3, 4, 20, 60]$ and the contextual states that is represented by C_t writes as follows:

$$C_t = \begin{pmatrix} c^1(t) & \dots & c^1(t-60) \\ \dots & \dots & \dots \\ c^3(t) & \dots & c^3(t-60) \end{pmatrix}$$

In contrast to asset states, contextual states C_t are only represented by a two dimensions tensor or equivalently a matrix. If we want to use convolutional networks, we therefore need to use 1D (for 1 dimensional) and not 2D (2 dimensional) convolutions. In addition, we add in these common contextual features the maximum portfolio strategy's return, the maximum and minimum portfolio strategy's volatilities. The latter two are like for asset states motivated by the stylized fact that standard deviations are useful features to detect crisis.

Last but not least we can also introduce that our state s_t incorporates the previous portfolio allocation. Hence our state can take the following three inputs:

- previous portfolio strategy returns lagged by δ_1 called the asset states A_t ;
- contextual features observed lagged by δ_2 called the common states C_t ;
- the previous weight allocation w_{t-1} ;

or mathematically, $S_t = \{A_t, C_t, w_{t-1}\}$

Our optimal control problem is to find the optimal policy $\pi^*(S_t)$ that maximizes the total reward denoted by $R(T)$ for one episode. Under very strong theoretical assumptions, this optimal policy always exists and is unique. In practice, we are far from the theoretical framework and we may find only locally optimal policies thanks to gradient ascent! The policy is represented by a deep network whose parameters are given by θ and composed of three sub-networks as illustrated in figure 3 and further described in II-A. Hence the optimal control problem writes as

$$\max_{\theta} \mathbf{E}_{\pi_{\theta}(\cdot)} R(T),$$

where $\mathbf{E}_{\pi_{\theta}}$ represents the expectation under the assumption that our policy $\pi_{\theta}(s_t)$ is precisely represented by our deep networks whose parameters are θ for a state at time t given by s_t . The total reward $R(T)$ can either be the net performance of the portfolio or some risk return criterion like the Sharpe ratio computed as the ratio of the average mean return over its standard deviation.

A. Network Architecture

Our network (as described in figure 3) uses three types of inputs:

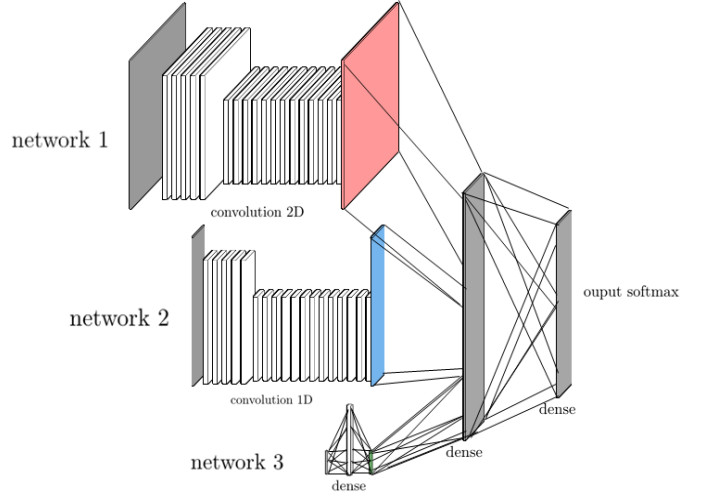


Fig. 3. Possible DRL network architecture

- sub-network 1: portfolio returns and standard deviations observed over the lag δ_1 array (the asset states A_t);
- sub-network 2: contextual information given by the correlation between equities and bonds, the Citigroup economic surprise and the risk aversion indexes observed over the lag δ_2 array and other additional common features like the maximum portfolio strategy's return, the maximum and minimum portfolio strategy's volatilities (the context states C_t);
- and potentially sub-network 3: the previous portfolio allocation w_{t-1}

We concatenate these 3 networks into a final one using two dense layers and a final softmax one to infer the portfolio weights.

Our reward is either the Sharpe ratio or the net value of the final portfolio. In terms of network internal architecture, we can either use convolution layers for sub-network 1 and 2 (convolution 2D and 1D respectively) or LSTM units. We can also do adversarial training by introducing some Gaussian noise in the training to make each iteration slightly different. This helps to have more robust models.

Concerning the train-validation-test split of our data-set, we use the following split: Train data-set is from 01-Jan-2010 to 31-Dec-2015, validation set from 01-Jan-2016 to 31-Dec-2017, while test data set ranges from 01-Jan-2018 to 31-Mar-2020. Hyper-parameters are tested on the validation sets. We provide the hyper-parameters used in the final run in table IV. Results are quite sensitive to the Adam learning rate and the lag 1 and 2 arrays. We tried various solutions and found that taking the last week of observation, the last month and the last quarter was working well and quite intuitive for the the lag 1 and 2 arrays. Results of the various trained networks and performance over iterations can be visualized in <http://www.aisquareconnect.com/deeprl/ICPRSummary.html> and are also given as supplementary materials of this paper.

All in all, the different possible network configurations and

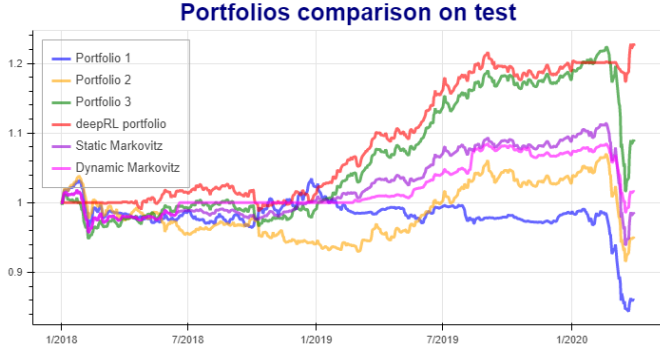


Fig. 4. Deep RL Portfolio Optimisation Result

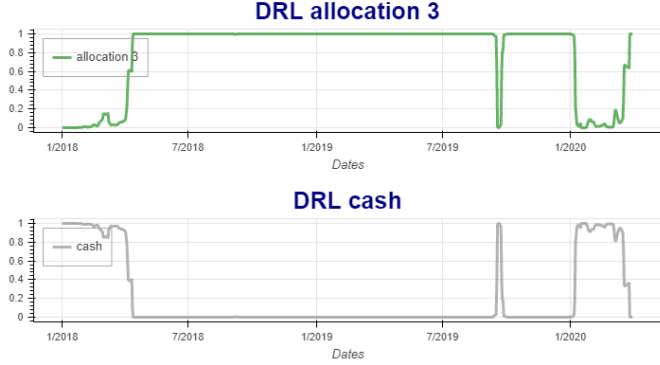


Fig. 5. Corresponding optimal allocation

architectures represent 32 models whose results are given in table II. In our experiment, $m = 4$ with the first three assets representing real strategies, while the fourth one being just cash whose value do not change over time. To represent the performance of each of the 3 strategies, we plot portfolio 1 which consists in taking only strategy 1 (in blue in figure 4), respectively portfolio 2 and 3 taking only strategy 2 and 3 (in orange and green).

It is worth noticing that the portfolio 3 consisting of 100 % in strategy 3 has a strong tendency to over-perform the other two strategies (portfolio 1 and 2). Hence we expect the deep RL agent to allocate mostly in strategy 3 and when anticipating a crisis, to allocate in cash. This is exactly what it does as illustrated in figure 5. It is also interesting to notice that the trained deep RL agent is mostly invested in strategy 3 and from time to time swap this allocation to a pure cash allocation. The anticipated crisis in 2018 enables the agent to slightly over-perform portfolio 3 from 2018 to the end of 2019. The agent however is not all mighty and makes mistake as illustrated by the wrong peaked cash allocation in end of 2019. It is able to adapt to the Covid crisis and to brutally swap allocation from strategy 3 to cash and back as markets bounced back at the end of March.

B. DRL algorithm

To find the optimal action $\pi^*(S_t)$ (in terms of portfolio allocation), we use deep policy gradient method with non

linear activation (Relu). We use buffer replay to memorize all marginal rewards, so that we can start batch gradient descent once we reached the final time step. We use the traditional Adam optimization so that we have the benefit of adaptive gradient descent with root mean square propagation [31].

C. Results

Performance results are given below in table II. Best performing models are highlighted in yellow. Returns are computed annually. Hence for a total performance of 21 % (as shown in figure 4) over the period of January 1st 2018 to March 31st 2020, the corresponding annual return is 8.8 %. Overall, out of the 32 models available, there are many DRL models that are able to over-perform not only traditional methods like static Markovitz but also the best portfolio (sometimes referred as the naive winner strategy) in terms of net performance and Sharpe ratio, with a final annual net return of 8.8% when using the best net profit reward model or 8.6% when using the best Sharpe ratio reward model compared to 3.9 % for the naive winner method. Dynamic Markovitz method consists in computing the Markovitz optimal allocation every 3 months. The Naive winner method consists in just selecting the best strategy over the train data set, which is strategy 3.

TABLE I
PERFORMANCE RESULTS

	Portfolio 1	Portfolio 2	Portfolio 3	Dynamic Markovitz	Deep RL Net_profit	Deep RL Sharpe	Naive winner
Net Performance	-6.3%	-2.1%	3.9%	0.7%	8.8%	8.6%	3.9%
Std dev	6.1%	6.5%	7.3%	4.3%	4.5%	4.2%	7.3%
Sharpe ratio	na	na	0.53	0.17	1.95	2.08	0.53

III. LEARNING OF THE NETWORK PARAMETERS

The agent's objective is to maximize its total reward R given at episode end. This reward can be net portfolio performance or Sharpe ratio computed as portfolio mean return over its standard deviation. Because we somehow play and play again the same scenario with the same reward function, the current framework has two important distinctions from many other RL problems. One is that the domain knowledge of the environment is well-mastered, and can be fully exploited by the agent. This exact expressiveness is a direct consequence that the agent's action has no influence on future price, which is clearly the case for small transactions or liquid assets. This isolation of action and external environment also allows one to use the same segment of market history to evaluate difference sequences of actions.

The second distinction is that the final reward depend on all episodic actions. In other words all episodic actions are important, justifying the full-exploitation approach.

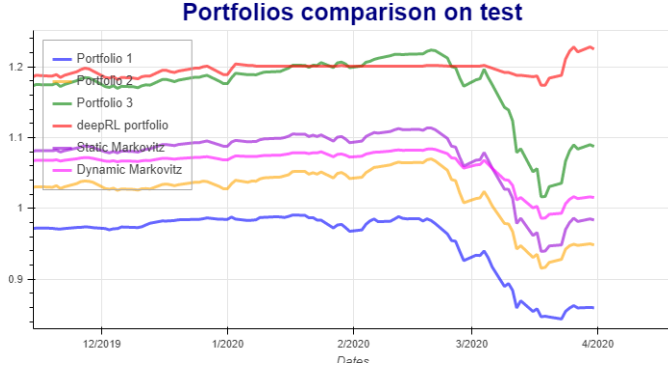


Fig. 6. Crisis portfolio reaction. It is worth noticing that the best DRL agent has a stable performance as it rapidly dis-invests and put asset in cash during the Covid crisis

A. Deterministic Policy Gradient

A policy is a mapping from the state space to the action space, $\pi : \mathcal{S} \rightarrow \mathcal{A}$. With full exploitation in the current framework, an action is deterministically produced by the policy from a state. The optimal policy is obtained using a gradient ascent algorithm. To achieve this, a policy is specified by a set of parameter θ , and $a_t = \pi_\theta(s_t)$. The performance metric of π_θ for time interval $[0, t]$ is defined as the corresponding reward function of the interval,

$$J_{[0,t]}(\pi_\theta) = R(s_1, \pi_\theta(s_1), \dots, s_t, \pi_\theta(s_t), s_{t+1}). \quad (1)$$

After random initialization, the parameters are continuously updated along the gradient direction with a learning rate λ ,

$$\theta \longrightarrow \theta + \lambda \nabla_\theta J_{[0,t]}(\pi_\theta). \quad (2)$$

To make the gradient ascent optimization, we use the standard Adam (short for Adaptive Moment Estimation) optimizer to have the benefit of adaptive gradient descent with root mean square propagation [31].

B. Crisis adaptation

It is remarkable that the DRL approach is able to handle the Covid crisis softly as displayed by figure 6. If we zoom over the period out of sample from December 2019 to March 2020, we can see that the DRL agent is able to rapidly reduce exposure to strategy 3 and allocate in cash, detecting thanks to contextual information that a crisis is imminent as show in figure 7. Interestingly, the DRL agent reallocates to portfolio 3 in March picking the market rebound.

Best networks as illustrated in table II are mostly convolutional networks. We found that for the sub network 1, it is optimal to have 2 convolutional layers but only 1 convolutional layer for the sub network 2. We illustrate the sub-network 1 in figure 8.

C. Impact of contextual information

Logically, networks with contextual information performs better as they have more information. For each network configuration, we compute the difference between the version with and without contextual information. We summarized

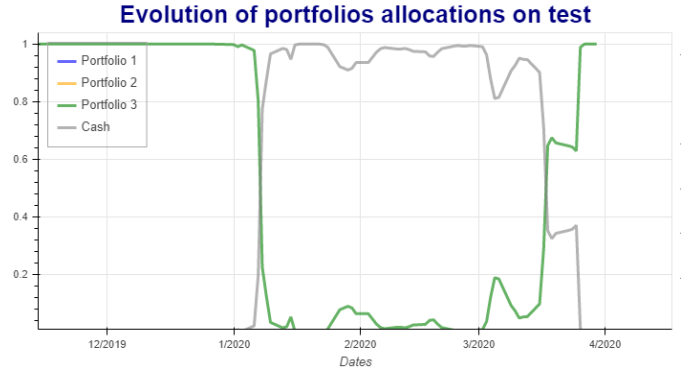


Fig. 7. Allocation during Covid crisis. It is worth noticing that the best DRL agent allocates almost all asset either in cash or strategy 3 and is rarely mixing the two strategies. It does not include at all strategies 1 or 2, indicating that the optimal choice is to saturate the allocation constraints as we only permit the dynamic agent to allocate between 0 and 100 %

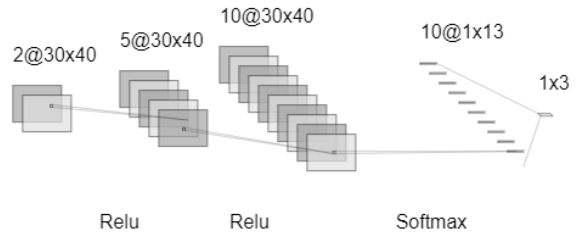


Fig. 8. convolutional network 1

these results in table III with best results highlighted in yellow. For all configurations, the version with contextual information achieves higher annual returns. This is almost the case also on Sharpe ratio, but there are exceptions. If we remove for each criterion the two largest difference classified as outliers, we found that contextual based models increase on average annual returns by 2.45 % and Sharpe ratio by 0.29.

IV. FURTHER WORK

On experiment, we see that the contextual based approach over-performs baseline methods like Markovitz. We also experienced that CNN architecture performs much better than LSTM units as they reduce the number of parameters to train and share parameters across portfolio strategies. Adversarial training makes also the training more robust by providing a more challenging environment. Last but not least, it is quite important to fine tune the numerous hyper-parameters of the contextual based DRL model, namely the various lags (lags period for the sub network fed by portfolio strategies past returns, lags period for common contextual features referred to as the common features in the paper), standard deviation period, learning rate, etc... It is compelling that the suggested framework is linearly scalable with the portfolio size and can accommodate contextual information. Our findings suggest that modeling the state with previous weight allocation

deteriorates training and does not help suggesting that the artifact of introducing previous weight to have a direct impact on state when performing an action is artificial and that in reality under the assumption of small market impact, it is more efficient to assume that portfolio allocation does not influence future state. The memory mechanism is quite beneficial as it allows to compute the final reward on each episode and hence allows avoiding gradient vanishing problem faced by many deep networks. Moreover, thanks to this memory mechanism, it is not challenging to create an online learning mechanism that can continuously digest incoming market information to improve the dynamic agent. The profitability of this framework surpasses traditional portfolio-selection methods, as demonstrated in the paper by a non negligible factor as it outperforms dynamic Markovitz by than 8 % and the best strategy by 4 %.

This better performance should be mitigated by the fact that the dynamic DRL agent is able to adapt well to the Covid crisis. Hence it benefits from an exceptional and almost unique condition in the financial history. Consequently, these numbers should not be taken literally but rather as a sign of the capacity of deep RL method to achieve human performance in portfolio allocation and to be able to detect and adapt to crisis patterns. Despite the efficiency of contextual based DRL models in experiments, these models can be improved in future works. Their main weakness is the number of hyper parameters that needs to be estimated on the validation set. Their second major weakness relies on the fact that in finance, each experience is somehow unique and one may not be able to draw conclusion on a single test set. Drawing a general conclusion is premature and beyond reason at this stage. It should be tested on more financial markets and on more outcomes. It may also be tested in terms of stability and capacity to adapt to further crisis patterns.

V. CONCLUSION

In this paper, we address the challenging task of detecting and adapting portfolio allocation to crisis environment. Our approach is based on deep reinforcement learning using contextual information thanks to a second sub-network. The model takes not only past performances of portfolio strategies over different rolling period, but also portfolio strategies standard deviation as well as contextual information like risk aversion, Citigroup economic surprise index, correlation between equity and bonds over a rolling period to make best allocation decision. The additional contextual information makes the learning of the dynamic asset manager agent more robust to crisis environment as the agent reacts more rapidly to changing environments. In addition, the usage of standard deviation of portfolio strategies provides a good hint for future crisis. The model achieves better performance than standard financial models. There are room for further improvement as this model constitutes only a first attempt to find a reasonable DRL solution to adapt to crisis situation and to answer positively if DRL can reach human level in applications to financial problems and in particular in detecting pattern crisis.

TABLE II
RESULTS OF THE VARIOUS MODELS

Reward	Adversarial training?	Network	Previous weight?	Context?	Annual return	Sharpe
NetProfit	No	Conv2D	No	Yes	8.8%	1.95
Sharpe	Yes	Conv2D	No	Yes	8.6%	2.08
NetProfit	No	Conv2D	Yes	Yes	8.5%	2.03
Sharpe	No	Conv2D	No	Yes	8.4%	2.01
NetProfit	Yes	Conv2D	No	Yes	8.0%	1.35
NetProfit	Yes	Conv2D	No	No	7.7%	1.94
Sharpe	No	Conv2D	No	No	6.4%	1.31
NetProfit	No	LSTM	No	Yes	6.2%	1.49
NetProfit	No	Conv2D	No	No	5.4%	0.97
Sharpe	Yes	LSTM	No	Yes	5.4%	1.23
NetProfit	Yes	LSTM	No	Yes	5.1%	0.93
NetProfit	Yes	Conv2D	Yes	Yes	4.3%	0.63
Sharpe	Yes	Conv2D	No	No	4.2%	0.69
NetProfit	No	LSTM	Yes	Yes	3.8%	0.52
Sharpe	No	Conv2D	Yes	No	3.8%	0.52
NetProfit	No	Conv2D	Yes	Yes	3.8%	0.52
Sharpe	Yes	LSTM	Yes	Yes	3.8%	0.52
Sharpe	Yes	Conv2D	Yes	Yes	3.7%	0.51
NetProfit	Yes	Conv2D	Yes	No	3.7%	0.51
NetProfit	No	LSTM	Yes	No	3.6%	0.49
NetProfit	Yes	LSTM	Yes	Yes	3.5%	0.48
NetProfit	Yes	LSTM	No	No	3.4%	1.24
Sharpe	No	LSTM	Yes	Yes	3.4%	0.48
NetProfit	No	LSTM	No	No	3.4%	0.47
Sharpe	Yes	Conv2D	Yes	No	3.4%	0.51
NetProfit	Yes	LSTM	Yes	No	2.3%	0.97
Sharpe	Yes	LSTM	No	No	2.3%	0.32
Sharpe	No	LSTM	No	Yes	1.5%	0.22
Sharpe	No	Conv2D	Yes	No	0.9%	0.13
Sharpe	Yes	LSTM	Yes	No	-5.1%	na
Sharpe	No	LSTM	No	No	-5.1%	na
Sharpe	No	LSTM	Yes	No	-5.1%	na

REFERENCES

- [1] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis, "Mastering the game of go without human knowledge," *Nature*, vol. 550, pp. 354–, Oct. 2017.
- [2] O. Vinyals, I. Babuschkin, W. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. Choi, R. Powell, T. Ewalds, P. Georgiev, J. Oh, D. Horgan, M. Kroiss, I. Danihelka, A. Huang, L. Sifre, T. Cai, J. Agapiou, M. Jaderberg, and D. Silver, "Grandmaster level in starcraft ii using multi-agent reinforcement learning," *Nature*, vol. 575, 11 2019.
- [3] S. Wang, D. Jia, and X. Weng, "Deep reinforcement learning for autonomous driving," *ArXiv*, vol. abs/1811.11329, 2018.
- [4] H. Markowitz, "Portfolio selection," *Journal of Finance*, vol. 7, pp. 77–91, 1952.
- [5] F. Black and R. Litterman, *Global portfolio optimization*. Financial Analysts, 1992.
- [6] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wier-

TABLE III
DIFFERENCE IN RETURNS AND SHARPE BETWEEN MODEL WITH AND WITHOUT CONTEXTUAL INFORMATION

Reward	Adversarial training?	Network	Previous weight?	Annual return difference	Sharpe difference
NetProfit	No	Conv2D	No	3.43%	0.99
NetProfit	No	Conv2D	Yes	4.68%	1.51
NetProfit	No	LSTM	No	2.77%	1.03
NetProfit	No	LSTM	Yes	0.27%	0.03
NetProfit	Yes	Conv2D	No	0.32%	- 0.59
NetProfit	Yes	Conv2D	Yes	0.58%	0.12
NetProfit	Yes	LSTM	No	1.70%	- 0.32
NetProfit	Yes	LSTM	Yes	1.16%	- 0.48
Sharpe	No	Conv2D	No	2.02%	0.70
Sharpe	No	Conv2D	Yes	2.94%	0.40
Sharpe	No	LSTM	No	6.56%	0.22
Sharpe	No	LSTM	Yes	8.52%	0.48
Sharpe	Yes	Conv2D	No	4.39%	1.39
Sharpe	Yes	Conv2D	Yes	0.36%	0.01
Sharpe	Yes	LSTM	No	3.05%	0.91
Sharpe	Yes	LSTM	Yes	8.87%	0.52

TABLE IV
HYPER PARAMETERS USED

hyper-parameters	value	description
batch size	50	Size of mini-batch during training
regularization coefficient	1e-8	L_2 regularization coefficient applied to network training
learning rate	0.01	Step size parameter in Adam
standard deviation period	20 days	period for standard deviation in asset states
commission	10 bps	commission rate
stride	2,1	stride used in convolution networks
conv number 1	5,10	number of convolutions in sub-network 1
conv number 2	2	number of convolutions in sub-network 2
lag period 1	[60, 20, 4, 3, 2, 1, 0]	lag period for asset states
lag period 2	[60, 20, 4, 3, 2, 1, 0]	lag period for contextual states
noise	0.002	adversarial Gaussian standard deviation

stra, and M. Riedmiller, "Playing atari with deep reinforcement learning," *NIPS Deep Learning Workshop*, 2013.

- [7] F. Freitas, A. De Souza, and A. Almeida, "Prediction-based portfolio optimization model using neural networks," *Neurocomputing*, vol. 72, pp. 2155–2170, 06 2009.
- [8] S. Niaki and S. Hoseinzade, "Forecasting s&p 500 index using artificial neural networks and design of experiments," *Journal of Industrial Engineering International*, vol. 9, 02 2013.
- [9] J. B. Heaton, N. G. Polson, and J. H. Witte, "Deep learning for finance: deep portfolios," *Applied Stochastic Models in Business and Industry*, vol. 33, no. 1, pp. 3–12, 2017.
- [10] Z. Jiang and J. Liang, "Cryptocurrency Portfolio Management with Deep Reinforcement Learning," *arXiv e-prints*, Dec. 2016.
- [11] Zhengyao et al., "Reinforcement learning framework for the financial portfolio management problem," *arXiv*, 2017.
- [12] Liang et al., "Adversarial deep reinforcement learning in portfolio

management," 2018.

- [13] P. Yu, J. S. Lee, I. Kulyatin, Z. Shi, and S. Dasgupta, "Model-based deep reinforcement learning for financial portfolio optimization," *RWSDM Workshop, ICML 2019*, 01 2019.
- [14] H. Wang and X. Y. Zhou, "Continuous-Time Mean-Variance Portfolio Selection: A Reinforcement Learning Framework," *arXiv e-prints*, Apr. 2019.
- [15] A. Gupta, R. Mendonca, Y. Liu, P. Abbeel, and S. Levine, "Meta-Reinforcement Learning of Structured Exploration Strategies," *arXiv e-prints*, Feb. 2018.
- [16] K. Lee, Y. Seo, S. Lee, H. Lee, and J. Shin, "Context-aware Dynamics Model for Generalization in Model-Based Reinforcement Learning," *arXiv e-prints*, May 2020.
- [17] J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, T. Lillicrap, and D. Silver, "Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model," *arXiv e-prints*, Nov. 2019.
- [18] L. Kaiser, M. Babaeizadeh, P. Milos, B. Osinski, R. H. Campbell, K. Czechowski, D. Erhan, C. Finn, P. Kozakowski, S. Levine, A. Mohiuddin, R. Sepassi, G. Tucker, and H. Michalewski, "Model-Based Reinforcement Learning for Atari," *arXiv e-prints and ICLR 2020*, Mar. 2019.
- [19] M. Zhang, S. Vikram, L. Smith, P. Abbeel, M. J. Johnson, and S. Levine, "SOLAR: Deep Structured Representations for Model-Based Reinforcement Learning," *arXiv e-prints and ICML 2019*, p. arXiv:1808.09105, Aug. 2018.
- [20] A. Nagabandi, I. Clavera, S. Liu, R. S. Fearing, P. Abbeel, S. Levine, and C. Finn, "Learning to Adapt in Dynamic, Real-World Environments Through Meta-Reinforcement Learning," *arXiv e-prints and ICLR 2019*, Mar. 2018.
- [21] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, "Dream to Control: Learning Behaviors by Latent Imagination," *arXiv e-prints and ICRL 2020*, Dec. 2019.
- [22] M. P. Deisenroth and C. E. Rasmussen, "Pilco: A model-based and data-efficient approach to policy search," in *In Proceedings of the International Conference on Machine Learning*, 2011.
- [23] S. Levine and P. Abbeel, "Learning neural network policies with guided policy search under unknown dynamics," in *Advances in Neural Information Processing Systems 27*, Z. Ghahramani, M. Welling, C. Cortes, N. D. Lawrence, and K. Q. Weinberger, Eds. Curran Associates, Inc., 2014, pp. 1071–1079.
- [24] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. The MIT Press, 2018.
- [25] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *ICML*, vol. 48. New York, New York, USA: PMLR, 20–22 Jun 2016, pp. 1928–1937.
- [26] M. Jaderberg, V. Mnih, W. M. Czarnecki, T. Schaul, J. Z. Leibo, D. Silver, and K. Kavukcuoglu, "Reinforcement learning with unsupervised auxiliary tasks," *CoRR*, vol. abs/1611.05397, 2016.
- [27] S. A. Ross, "The arbitrage theory of capital asset pricing," *Journal of Economic Theory*, vol. 13, no. 3, pp. 341–360, 1976.
- [28] D. Harmon, B. Stacey, Y. Bar-Yam, and Y. Bar-Yam, "Networks of Economic Market Interdependence and Systemic Risk," *arXiv e-prints*, Nov. 2010.
- [29] F. Black, "Studies of stock price volatility changes," *Proceedings of the 1976 Meetings of the American Statistical Association, Business and Economical Statistics Section*, 1976.
- [30] G. Wu, "The Determinants of Asymmetric Volatility," *Review of Financial Studies*, vol. 14, no. 3, pp. 837–859, 2001.
- [31] D. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014.